

PySpark + Delta Lake

COMPLETE TRANSFORMATION & OPTIMIZATION CHEAT SHEET

SELECT & RENAME

Select columns	<code>df.select("a", "b", "c")</code>
Select with expr	<code>df.selectExpr("a * 2 as a2")</code>
Rename column	<code>df.withColumnRenamed("old", "new")</code>
Drop column	<code>df.drop("col")</code>
Add / transform	<code>df.withColumn("c", col("a") + 1)</code>
Cast type	<code>col("age").cast("integer")</code>
Conditional col	<code>when(col("x")>0,"pos").otherwise("neg")</code>

WINDOW FUNCTIONS

DEFINE WINDOW	
<pre>from pyspark.sql import Window w = Window.partitionBy("dept").orderBy("salary")</pre>	
Row number	<code>row_number().over(w)</code>
Rank / dense rank	<code>rank().over(w) / dense_rank().over(w)</code>
Lag / Lead	<code>lag("col",1).over(w) / lead("col",1).over(w)</code>
Running total	<code>sum("v").over(w.rowsBetween(-sys.maxsize, 0))</code>
ntile	<code>ntile(4).over(w) # quantiles</code>

Z-ORDER OPTIMIZATION

OPTIMIZE + ZORDER	
<pre>spark.sql(""" OPTIMIZE delta.`/path/to/table` ZORDER BY (country, event_date) """)</pre>	
✓ WHEN TO USE High-cardinality cols in WHERE/JOIN. 2-4 cols max. Co-locates data → fewer files scanned.	
⚠ AVOID ON Partition cols, boolean/low-cardinality cols, cols never used in filters.	

FILTER & WHERE

Simple filter	<code>df.filter(col("age") > 18)</code>
String condition	<code>df.where("age > 18 AND city = 'NYC'")</code>
AND / OR	<code>(col("a")>1) & (col("b")="x")</code>
isin	<code>col("c").isin(["a", "b", "c"])</code>
Like / contains	<code>col("name").like("%John%")</code>
startsWith	<code>col("name").startswith("J")</code>
Null checks	<code>col("x").isNull() / .isNotNull()</code>

SORT, DEDUP & NULLS

Sort asc / desc	<code>df.orderBy(col("a").desc())</code>
Multi-col sort	<code>df.orderBy("a", col("b").desc())</code>
Distinct rows	<code>df.distinct()</code>
Drop duplicates	<code>df.dropDuplicates(["a", "b"])</code>
Fill nulls	<code>df.fillna({"a":0, "b":"N/A"})</code>
Drop null rows	<code>df.dropna(subset=["a", "b"])</code>
Replace values	<code>df.replace("old", "new", "col")</code>

VACUUM — STORAGE CLEANUP

RUN VACUUM	
<pre># Dry run first spark.sql("VACUUM delta.`/path` DRY RUN") # Default = 168 h (7 days) spark.sql("VACUUM delta.`/path/to/table`") # Custom retention spark.sql("VACUUM delta.`/path` RETAIN 168 HOURS")</pre>	
⚠ CAUTION Never below 168 h – breaks time travel. Never below streaming checkpoint lag.	

JOINS

Inner join	<code>df1.join(df2, "id", "inner")</code>
Left outer	<code>df1.join(df2, "id", "left")</code>
Full outer	<code>df1.join(df2, "id", "outer")</code>
Multi-key join	<code>df1.join(df2, ["a","b"], "inner")</code>
Broadcast join	<code>df1.join(broadcast(df2), "id")</code>
Left anti	<code>df1.join(df2, "id", "left_anti")</code>
Semi join	<code>df1.join(df2, "id", "left_semi")</code>

PARTITION & COALESCE

Repartition (shuffle)	<code>df.repartition(200)</code>
Repartition by col	<code>df.repartition(50, col("country"))</code>
Coalesce (no shuffle)	<code>df.coalesce(10)</code>
Check partitions	<code>df.rdd.getNumPartitions()</code>
Set shuffle parts	<code>spark.conf.set("spark.sql.shuffle.partitions", "200")</code>
✓ RULE OF THUMB repartition to increase/balance · coalesce to reduce before write · ~128 MB per partition	

GROUPBY & AGGREGATE

Count	<code>df.groupBy("col").count()</code>
Sum / avg / min / max	<code>.agg(sum("v"), avg("v"), min("v"))</code>
Alias result	<code>.agg(sum("v").alias("total"))</code>
Count distinct	<code>.agg(countDistinct("id"))</code>
Collect list / set	<code>.agg(collect_list("x"), collect_set("x"))</code>
First / last	<code>.agg(first("x"), last("x"))</code>
Pivot	<code>.groupBy("a").pivot("b").sum("v")</code>

UNION & SET OPS

Union by position	<code>df1.union(df2)</code>
Union by name	<code>df1.unionByName(df2)</code>
Allow missing cols	<code>.unionByName(df2, allowMissingColumns=True)</code>
Intersect	<code>df1.intersect(df2)</code>
Subtract	<code>df1.subtract(df2)</code>

DELTA LAKE — WRITE, MERGE & TIME TRAVEL

WRITE MODES	MERGE / UPSERT	TIME TRAVEL	HISTORY & SCHEMA
<pre>df.write.format("delta") \ .mode("overwrite").save(path) # overwrite append ignore error .partitionBy("date").save(path)</pre>	<pre>from delta.tables import DeltaTable tgt = DeltaTable.forPath(spark, path) tgt.alias("t").merge(src.alias("s"), "t.id = s.id") \ .whenMatchedUpdateAll() \ .whenNotMatchedInsertAll() \ .execute()</pre>	<pre># by version spark.read.format("delta") \ .option("versionAsOf", 3) \ .load(path) # by timestamp .option("timestampAsOf", "2024-01-01") tgt.restoreToVersion(2)</pre>	<pre>tgt.history().show() # audit log # merge new columns on write .option("mergeSchema", "true") # force overwrite schema .option("overwriteSchema", "true")</pre>

PERFORMANCE PATTERNS & KEY CONFIGS

BROADCAST JOIN	CACHE & PERSIST	COMMON PITFALLS	KEY SPARK CONFIGS
<pre>from pyspark.sql.functions import broadcast df1.join(broadcast(small_df), "id") # one side < 10 MB → no shuffle</pre>	<pre>df.cache() # MEMORY_AND_DISK df.persist(StorageLevel.MEMORY_ONLY) df.unpersist() # free when done</pre>	<pre># ✗ .collect() on large df # ✗ Python UDFs (use built-ins) # ✗ .count() inside loops # ✓ df.limit(100).show() # ✓ df.explain() – check plan</pre>	<pre>shuffle.partitions = 200 autoBroadcastJoinThreshold = 10m adaptive.enabled = true delta.optimizeWrite = true # prefix: spark.sql.* / spark.databricks.*</pre>